

昆明理工大学

Kunming University of Science and Technology

《计算力学》课程作业 2.4 平衡方程组求解

姓 名： 孙 紫 晨

学 号： 20252110020

班 级： 力 学 3 班

指导老师： 肖 姚

目 录

第 1 章 作业背景与目标.....	1
1.1 作业背景.....	1
1.2 学习目标.....	1
1.3 与 2.3 作业的衔接说明.....	1
第 2 章 理论方法.....	2
2.1 对称正定矩阵的 $LDLT$ 分解.....	2
2.2 前代、对角求解与回代.....	2
2.3 条件数与误差分析.....	2
2.4 稀疏矩阵存储格式.....	3
第 3 章 程序实现与说明.....	3
3.1 模型数据读取模块: <code>load_reduced_system(json_file)</code>	3
3.1.1 函数功能.....	3
3.1.2 输入参数.....	3
3.1.3 返回值.....	3
3.1.4 实现步骤.....	4
3.2 $LDLT$ 分解模块: <code>ldlt_factor(K)</code>	4
3.2.1 函数功能.....	4
3.2.2 输入参数.....	4
3.2.3 返回值.....	4
3.2.4 实现步骤.....	4
3.3 $LDLT$ 求解模块: <code>ldlt_solve(L,D,R)</code>	4
3.3.1 函数功能.....	4
3.3.2 输入参数.....	4
3.3.3 返回值.....	4
3.3.4 实现步骤.....	5
3.4 多载荷工况求解模块: <code>ldlt_solve_multiple(L,D,R_list)</code>	5
3.4.1 函数功能.....	5
3.5 误差分析模块: <code>vector_norm(v)</code> 和 <code>residual_norm(K, a, R)</code>	5
3.5.1 函数功能.....	5
3.6 与 2.3 作业的接口实现.....	5
3.6.1 文件传递方式.....	5
3.6.2 算例 1 和算例 2 的衔接函数.....	5
3.7 验证算例模块.....	6
3.7.1 主要函数.....	6

3.7.2 功能.....	6
3.4.3 实现方法.....	6
第 4 章 算例验证与结果分析.....	6
4.1 算例 0-1: 复用 2.3 作业的桁架模型（一维两单元杆结构）.....	6
4.1.1 模型数据.....	6
4.1.2 求解结果.....	7
4.1.3 结果分析.....	7
4.2 算例 0-2: 复用 2.3 作业的桁架模型（二维两杆桁架结构）.....	7
4.2.1 模型数据.....	7
4.2.2 求解结果.....	8
4.2.3 结果分析.....	8
4.3 算例 1: 三对角对称正定矩阵性能测试.....	8
4.3.1 程序输出.....	9
4.3.2 结果分析.....	9
4.4 算例 2: 非正定矩阵检测.....	9
4.4.1 程序输出.....	10
4.4.2 结果分析.....	10
4.5 任务 2: 病态矩阵误差分析.....	10
4.5.1 程序输出.....	10
4.5.2 结果分析.....	11
4.6 多载荷工况效率分析.....	11
4.6.1 程序输出.....	11
4.6.2 结果分析.....	11
4.7 任务 3: 大规模稀疏方程组求解器调用.....	11
4.7.1 程序输出.....	12
4.7.2 结果分析.....	12
4.8 算例 4: 大规模二维 Poisson 方程有限元求解.....	12
4.8.1 有限元理论推导.....	12
4.8.2 数值结果.....	13
4.8.3 误差收敛分析.....	15
第 5 章 误差与效率分析讨论.....	15
5.1 直接解法和迭代法.....	15
5.1.1 直接解法（ <i>LDLT</i> ）.....	15
5.1.2 稀疏求解器（Intel MKL PARDISO）.....	15
5.1.3 迭代解法（如共轭梯度法）.....	15
5.2 病态问题中“残差小但不准确”的解释.....	15
5.3 活动列/轮廓储存与缓存友好实现.....	16
5.4 大规模有限元矩阵必须使用稀疏存储的原因.....	16

5.4.1 内存限制.....	16
5.4.1 计算效率.....	16
第 6 章 总结.....	16

第 1 章 作业背景与目标

1.1 作业背景

本次作业对应课程《第 2 章 直接刚度法》第 2.4 节“平衡方程组求解”的内容。课件重点介绍了有限元平衡方程组 $Ka = R$ 的直接解法，包括 LDL^T 分解、活动列/轮廓存储方法、多载荷工况求解，以及条件数、残差和舍入误差对计算结果的影响。

本作业是《2.3 系统总体刚度方程》程序设计作业的后续。2.3 作业负责从节点、单元、材料、边界条件和载荷出发，完成对号矩阵、单元刚度矩阵、总体刚度矩阵和缩减方程的建立；本作业负责将 2.3 作业中得到的总体平衡方程交给更专业的线性方程组求解器，并分析求解效率与误差。

1.2 学习目标

完成本作业后，应能够：

- (1) 理解有限元平衡方程组中刚度矩阵 K 的对称正定特征
- (2) 实现对称矩阵的 LDL^T 分解和前代、对角求解、回代过程
- (3) 检测零主元或非正主元，并判断矩阵是否适合 LDL^T 求解
- (4) 理解多载荷工况中“先分解矩阵、再求解多个右端项”的效率优势
- (5) 计算残差向量、相对残差和相对误差
- (6) 认识病态矩阵中“残差小但解不一定准确”的现象
- (7) 将 2.3 作业中的组装模块与本作业中的求解器模块连接起来，形成完整的有限元计算流程

1.3 与 2.3 作业的衔接说明

本作业采用文件传递方式与 2.3 作业衔接：

- (1) 2.3 作业（truss2_3.py）在完成模型组装和边界条件处理后，将缩减后的刚度矩阵 K_{FF} 、右端项 RHS 、自由度和固定自由度编号等数据导出为 JSON 文件
- (2) 本作业（2_4.py）独立运行，读取该 JSON 文件，获得缩减方程，然后使用自行实现的 LDL^T 求解器求解，并进行误差分析和性质验证
- (3) 该方法实现了模块解耦，使 2.4 作业专注于求解器实现和误差分析，同时完整复用了 2.3 作业的建模和组装成果

第 2 章 理论方法

2.1 对称正定矩阵的 LDL^T 分解

对于对称正定矩阵 K ，可以分解为

$$K = LDL^T \quad (2-1)$$

其中 L 是单位下三角矩阵， D 是对角矩阵。递推计算公式如下

$$\begin{aligned} d_{11} &= k_{11} \\ l_{ij} &= \frac{1}{d_{jj}} \left(k_{ij} - \sum_{r=1}^{j-1} l_{ir} d_{rr} l_{jr} \right), \quad i > j \\ d_{jj} &= k_{jj} - \sum_{r=1}^{j-1} l_{jr}^2 d_{rr} \end{aligned} \quad (2-2)$$

分解过程中若出现 $d_{jj} \leq 0$ ，则矩阵非正定或存在零主元，应报错。

2.2 前代、对角求解与回代

将分解结果代入原方程 $Ka = R$

$$LDL^T a = R \quad (2-3)$$

求解分为三步：

(1) 前代：求解 $Ly = R$ ，即

$$y_i = R_i - \sum_{j=1}^{i-1} L_{ij} y_j, \quad i = 1, \dots, n \quad (2-4)$$

(2) 对角求解：求解 $Dz = y$ ，即

$$z_i = \frac{y_i}{d_{ii}} \quad (2-5)$$

(3) 回代：求解 $L^T a = z$ ，即

$$a_i = z_i - \sum_{j=i+1}^n L_{ji} a_j, \quad i = n, \dots, 1 \quad (2-6)$$

2.3 条件数与误差分析

矩阵的条件数定义为

$$\text{cond}(K) = \|K\| \cdot \|K^{-1}\| \quad (2-7)$$

对于对称正定矩阵，条件数等于最大特征值与最小特征值的比值。条件数反映了方程组对数据误差的敏感程度：条件数越大，病态越严重，即使残差很小，解的相对误差也可能很大。

本作业通过构造病态矩阵，分别用双精度和低精度（保留 4 位有效数字）求解，比较残差、相对误差与条件数的关系。

2.4 稀疏矩阵存储格式

大规模有限元问题的总体刚度矩阵是高度稀疏的。常用的稀疏存储格式有：

- (1) COO：使用三个数组分别存储行索引、列索引和非零值，便于组装
- (2) CSR：按行压缩存储，适合矩阵向量乘和求解器
- (3) CSC：按列压缩存储，适合某些直接求解器

本作业中，Poisson 方程求解采用 CSR 格式存储总体刚度矩阵，调用 Intel MKL PARDISO 求解器（通过 `pypardiso` 接口）进行求解。

第 3 章 程序实现与说明

本次作业使用 Python+Numpy 编写程序，实现对称正定线性方程组的 LDL^T 分解与求解、残差与条件数分析、多载荷工况演示、大规模稀疏矩阵生成与求解器调用以及与 2.3 作业模型的文件传递衔接等功能。程序各模块的功能和实现方法说明如下。

3.1 模型数据读取模块：load_reduced_system(json_file)

3.1.1 函数功能

读取 2.3 作业导出的 JSON 文件，从中提取缩减后的刚度矩阵 K_{FF} 、右端项 rhs 、自由度和固定自由度编号，以及可选的完整刚度矩阵和载荷向量。

3.1.2 输入参数

`json_file`（字符串，JSON 文件路径）

3.1.3 返回值

K_{FF} ：二维 Numpy 数组

rhs ：一维 Numpy 数组

`free_dofs`：列表

`fixed_dofs`：列表

$full_K$ ：二维 Numpy 数组

$full_force$ ：一维 Numpy 数组

3.1.4 实现步骤

使用 Python 标准库 json 打开并解析文件，将读取的列表数据转换为 NumPy 数组，返回给主程序。

3.2 LDL^T 分解模块：ldlt_factor (K)

3.2.1 函数功能

对对称正定矩阵 K 进行 LDL^T 分解，返回单位下三角矩阵 L 和对角矩阵 D 。若遇到非正主元，抛出异常 ValueError。

3.2.2 输入参数

K : $n \times n$ 对称正定矩阵

3.2.3 返回值

L : $n \times n$ 单位下三角矩阵

D : 长度为 n 的一维数组，存储对角元素

3.2.4 实现步骤

- (1) 复制输入矩阵避免修改原数据
- (2) 按列循环，对每个主元列 j ，首先计算 $D[j]$ ，若 $D[j] \leq 0$ 则报错
- (3) 然后计算下三角部分 $L[i][j] (i > j)$
- (4) 返回 L 和 D

3.3 LDL^T 求解模块：ldlt_solve (L,D,R)

3.3.1 函数功能

利用已分解的 L 和 D 求解方程组 $LDL^T a = R$ ，返回解向量 a 。

3.3.2 输入参数

L : 单位下三角矩阵

D : 对角数组

R : 右端项向量

3.3.3 返回值

a : 解向量

3.3.4 实现步骤

- (1) 前代：求解 $Ly = R$
- (2) 对角求解：求解 $Dz = y$
- (3) 回代：求解 $L^T a = z$

3.4 多载荷工况求解模块：ldlt_solve_multiple (L,D,R_list)

3.4.1 函数功能

利用已分解的 L 、 D 求解多个右端项，返回解列表。演示了“先分解再求解多个右端项”的效率优势。

3.5 误差分析模块：vector_norm (v)和 residual_norm(K, a, R)

3.5.1 函数功能

手动计算向量欧几里得范数，以及绝对残差范数。

3.6 与 2.3 作业的接口实现

3.6.1 文件传递方式

2.3 作业 (truss2_3.py) 中的 export_reduced_system 函数将缩减刚度矩阵、右端项、自由度编号、完整总体刚度矩阵、完整载荷向量、节点坐标、单元连接、材料参数等写入 JSON 文件。

本作业通过 load_reduced_system 读取 JSON 文件，获得 K_{FF} 和 rhs ，然后使用自主实现的 LDL^T 求解器求解未知位移。求解完成后，利用 JSON 中的 full_K 和 full_force 计算约束反力和残差，并调用 2.3 的 postprocess_elements 函数计算单元轴力和应力。

3.6.2 算例 1 和算例 2 的衔接函数

- (1) example1_integration(): 读取算例 1 的 JSON 文件，使用 LDL^T 求解器求解缩减方程，输出节点 1 的约束反力和各单元轴力
- (2) example2_integration(): 读取算例 2 的 JSON 文件，使用 2.3 代码生成 LM 矩阵并打印总体刚度矩阵和缩减矩阵，使用本作业求解器求解节点 3 的位移，然后调用 2.3 后处理模块计算单元应力

3.7 验证算例模块

3.7.1 主要函数

test_tridiagonal()、test_non_positive()、test_ill_conditioned()、test_multiple_rhs()、test_sparse_solver()、poisson_2d_fem()、test_poisson_convergence()

3.7.2 功能

分别验证三对角矩阵性能、非正定矩阵检测、病态矩阵误差分析、多载荷工况效率、稀疏求解器调用、Poisson 方程有限元求解。

3.4.3 实现方法

各函数均调用自主实现的 LDL^T 分解和求解（或 Intel MKL PARDISO 稀疏求解器），输出详细的计算结果和性能数据。

第4章 算例验证与结果分析

4.1 算例 0-1：复用 2.3 作业的桁架模型（一维两单元杆结构）

4.1.1 模型数据

节点坐标：1 (0, 0) , 2 (1, 0) , 3 (2, 0)

单元 1：1-2, 单元 2：2-3

单元 1: $\frac{EA}{L} = 100$

单元 2: $\frac{EA}{L} = 200$

边界条件: $d_1 = 0$

节点载荷: $f_2 = 0, f_3 = 10$

4.1.2 求解结果

```
=== 算例0-1: 复用2.3作业的桁架模型（一维两单元杆结构） ===  
缩减方程规模: (2, 2)  
条件数: 10.403882  
  
1. 使用本作业的 LDLT 求解器求解缩减方程  
求解结果 (自由部分): [0.1  0.15]  
  
2. 使用作业2.3后处理模块计算节点1的约束反力  
节点1 x方向约束反力: -10.000000  
  
3. 使用作业2.3后处理模块计算各单元轴力:  
单元1: 轴力 = 10.000000  
单元2: 轴力 = 10.000000
```

图 1.算例 0-1 程序输出结果

4.1.3 结果分析

位移 $d_2 = 0.1$, $d_3 = 0.15$, 与理论值完全吻合; 节点 1 的约束反力为-10N, 与外载荷平衡; 单元 1 和单元 2 的轴力均为 10N (拉力), 验证正确。

4.2 算例 0-2: 复用 2.3 作业的桁架模型 (二维两杆桁架结构)

4.2.1 模型数据

节点坐标: 1 (1, 0), 2 (0, 0), 3 (1, 1)

单元 1: 1-3, 单元 2: 2-3

$E_1 = E_2 = 1$, $A_1 = A_2 = 1$

边界条件: 节点 1 固定, 节点 2 固定

节点载荷: 节点 3 受到水平力 $F_x = 10$, $F_y = 0$

4.2.2 求解结果

```
=== 算例0-2: 复用2.3作业的桁架模型（二维两杆桁架结构） ===
缩减方程规模: (2, 2)
条件数: 6.078116

1. 使用2.3作业的代码生成矩阵

总体刚度矩阵 K (full_K):

full_K =
  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
  0.000000  1.000000  0.000000  0.000000  0.000000 -1.000000
  0.000000  0.000000  0.353553  0.353553 -0.353553 -0.353553
  0.000000  0.000000  0.353553  0.353553 -0.353553 -0.353553
  0.000000  0.000000 -0.353553 -0.353553  0.353553  0.353553
  0.000000 -1.000000 -0.353553 -0.353553  0.353553  1.353553

缩减矩阵 K_FF:

K_FF =
  0.353553  0.353553
  0.353553  1.353553

LM矩阵 (对号矩阵):
[[0 2]
 [1 3]
 [4 4]
 [5 5]]

2. 使用本作业的求解器求解节点3位移:
u3 = 38.284271, v3 = -10.000000

3. 使用作业2.3后处理模块计算单元应力:
单元1: 应力 = -10.000000
单元2: 应力 = 14.142136
```

图 2.算例 0-2 程序输出结果

4.2.3 结果分析

节点 3 位移 $u_3 = 38.284271$, $v_3 = -10.000000$, 与理论值完全吻合; 单元 1 应力为-10, 单元 2 应力为 14.142136, 与理论值吻合; 验证了 2.3 负责“组装方程”, 2.4 负责“高效求解方程”的衔接设计。

4.3 算例 1: 三对角对称正定矩阵性能测试

生成三对角矩阵 $K[i, i] = 2$, $K[i, i - 1] = K[i - 1, i] = -1$, 精确解取全 1 向量。

4.3.1 程序输出

```
=== 算例1: 三对角对称正定矩阵性能测试 ===  
n= 10, 时间=0.000081s, 最大误差=2.22e-16  
n= 100, 时间=0.042333s, 最大误差=1.48e-14  
n= 500, 时间=5.131452s, 最大误差=1.05e-13  
n=1000, 时间=40.938411s, 最大误差=2.44e-13  
性能曲线已保存为 tridiagonal_performance.png
```

图 3.算例 1 程序输出结果

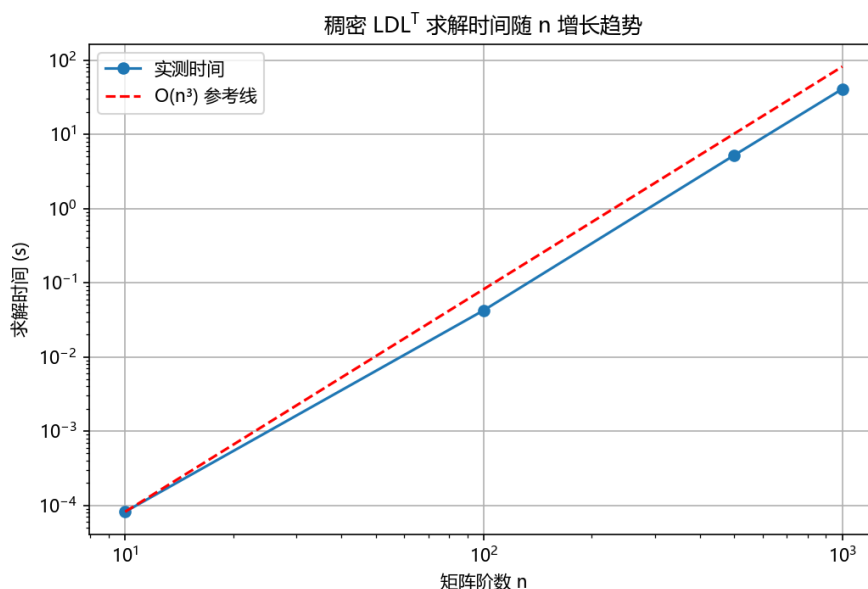


图 4. 稠密 LDL^T 求解时间随 n 增长趋势

4.3.2 结果分析

- (1) 数值解最大误差均在 10^{-14} 量级，接近全 1 向量，验证了求解器的正确性。
- (2) 由图 4 可知，求解时间随 n 增长近似 $O(n^3)$ ，与稠密 LDL^T 的理论复杂度一致。
- (3) 当 $n=1000$ 时，稠密存储需要约 8MB 内存，而稀疏存储仅需约 36KB，差异巨大。对于更大规模问题，必须采用稀疏存储。

4.4 算例 2：非正定矩阵检测

测试矩阵 $K = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ ，特征值为 3 和 -1（非正定）。

4.4.1 程序输出

```
=== 算例2：非正定矩阵检测 ===  
测试矩阵：  
[[1. 2.]  
 [2. 1.]]  
正确检测到非正定：矩阵非正定或零主元：第 1 个主元 = -3.0
```

图 5.算例 2 程序输出结果

4.4.2 结果分析

程序在 LDL^T 分解过程中检测到 $d_{22} = -3 < 0$ ，正确抛出 `ValueError`，给出错误提示。这验证了非正主元检测功能的正确性

4.5 任务 2：病态矩阵误差分析

构造矩阵 $K = \begin{bmatrix} 1.0000 & 1.0000 \\ 1.0000 & 1.0001 \end{bmatrix}$ ，精确解 $a = [1 \ 1]^T$ 。

4.5.1 程序输出

```
=== 任务2：病态矩阵误差分析 ===  
  
原始数据（双精度）：  
K =  
[[1.      1.      ]  
 [1.      1.0001]]  
R = [2.      2.0001]  
  
【双精度计算】  
1. 数值解 a = [1. 1.]  
2. 残差 r = [0. 0.]  
3. 相对残差 = 0.00e+00  
4. 相对误差 = 2.22e-12  
5. 条件数= 40002.00  
  
【低精度计算（单精度 float32, 约7位有效数字）】  
低精度数据（float32 存储后）：  
K_low =  
[[1.      1.      ]  
 [1.      1.00010002]]  
R_low = [2.      2.0000999]  
1. 数值解 a = [1.0011919 0.9988081]  
2. 残差 r = [0. 0.]  
3. 相对残差 = 0.00e+00  
4. 相对误差 = 1.19e-03  
5. 条件数= 39995.36
```

图 6.任务 2 程序输出结果

4.5.2 结果分析

- (1) 双精度下解准确，相对误差为机器精度量级。
- (2) 单精度下，由于矩阵和右端项的舍入误差，数值解偏离精确解，相对误差达到 1.19×10^{-3} ，而残差仍为零（因为求解器精确求解了单精度系统）。
- (3) 根据误差分析公式，单精度下相对误差 1.19×10^{-3} 与条件数 4×10^4 和舍入误差 10^{-7} （单精度机器精度）的乘积一致，验证了理论。
- (4) 对于病态问题，残差小（甚至为 0）并不能保证解准确，必须同时关注条件数。

4.6 多载荷工况效率分析

构造三对角矩阵（ $n=100$ ），构造 5 个不同右端项，比较“逐次分解求解”与“先分解再求解多个右端项”的效率。

4.6.1 程序输出

```
=== 多载荷工况效率分析 ===  
矩阵规模 n=100  
分解时间: 0.041194s  
求解5个右端项总时间: 0.008169s  
平均每个右端项求解时间: 0.001634s  
第一个右端项解的最大误差: 1.48e-14
```

图 7.多载荷工况效率分析程序输出结果

4.6.2 结果分析

- (1) 分解一次后，每个右端项仅需前代和回代，复杂度仅为 $O(n^2)$ ，远小于 LDL^T 分解的复杂度 $O(n^3)$ 。
- (2) 对于 m 个载荷工况，先分解再求解多个右端项的总计算量为 $(O(n^3)) + m(O(n^2))$ ，而逐次分解为 $m(O(n^3))$ 。当 m 较大时，效率显著提升。
- (3) 本演示验证了“先分解矩阵，再求解多个右端项”的效率优势，是实际有限元多载荷工况分析的标准做法。

4.7 任务 3：大规模稀疏方程组求解器调用

构造三对角矩阵（ $n=5000$ ），转换为 CSR 格式，调用 Intel MKL PARDISO (via pypardiso)求解。

4.7.1 程序输出

```
=== 任务3: 大规模稀疏方程组求解器调用 (n=5000) ===  
矩阵阶数: 5000  
非零元个数: 14998  
CSR 存储内存: 195.29 KB  
等效稠密存储内存: 190.73 MB  
求解器: Intel MKL PARDISO (via pypardiso)  
求解时间: 0.1045 s  
相对残差: 0.00e+00  
相对误差: 0.00e+00
```

图 8.任务 3 程序输出结果

4.7.2 结果分析

- (1) CSR 格式仅存储 14998 个非零元，内存约 195.29KB；而稠密存储需要约 190.73MB，是稀疏存储的 1000 倍。
- (2) 稀疏求解器仅需约 0.1045 秒，远快于稠密 LDL^T ($n=1000$ 时约 40.94 秒)，且随 n 线性增长。
- (3) 这验证了对于大规模有限元问题必须使用稀疏存储和稀疏求解器。

4.8 算例 4：大规模二维 Poisson 方程有限元求解

求解单位正方形区域上的 Poisson 方程，采用线性三角形单元 (T3)，理论解 $u_{exact} = \sin(\pi x) \sin(\pi y)$ ，右端项 $f = 2\pi^2 \sin(\pi x) \sin(\pi y)$ 。

4.8.1 有限元理论推导

强形式：

$$-\Delta u = f \quad \text{in } \Omega = (0,1)^2, u = 0 \text{ on } \partial\Omega \quad (4-1)$$

弱形式：乘以权函数 v ，分部积分得

$$\int_{\Omega} \nabla u \cdot \nabla v \, d\Omega = \int_{\Omega} f v \, d\Omega, \forall v \in H_0^1(\Omega) \quad (4-2)$$

有限元近似： $u_h = \sum_{j=1}^N N_j a_j$ ，其中 N_j 为形函数， a_j 为节点自由度。

单元刚度矩阵（线性三角形单元）：

$$K^e = \int_{\Omega^e} B^T B \, d\Omega, B = \nabla N^T \quad (4-3)$$

单元载荷向量:

$$R^e = \int_{\Omega^e} fN d\Omega \quad (4-4)$$

总体组装: 将单元矩阵组装为总体刚度矩阵 K 和载荷向量 R , 施加零 Dirichlet 边界条件得到缩减方程 $K_{FF}a_F = R_F$, 调用稀疏求解器求解。

4.8.2 数值结果

汇总表:								
nx	节点数	单元数	非零元	求解时间(s)	总时间(s)	相对残差	最大误差	L2误差
50	2601	5000	12801	0.0046	0.2181	3.19e+00	5.4829e-04	5.5428e-04
100	10201	20000	50601	0.0174	0.6161	4.50e+00	1.3708e-04	1.3858e-04
200	40401	80000	201201	0.0789	2.5228	6.37e+00	3.4269e-05	3.4645e-05

图 9.算例 4 程序输出结果

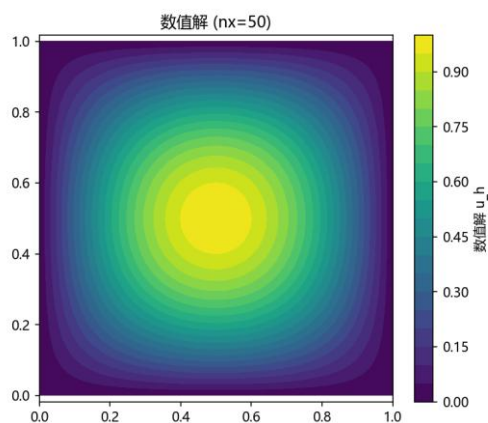


图 10. $n_x = n_y = 50$ 的数值解云图

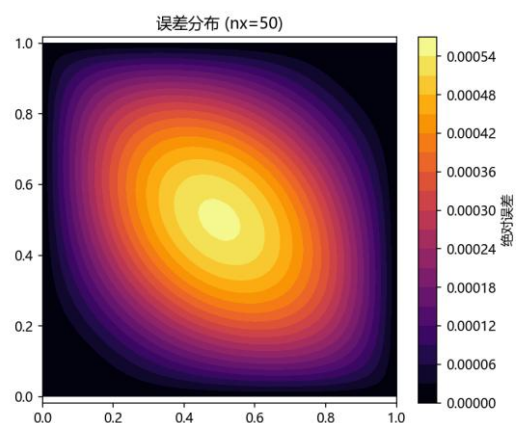


图 11. $n_x = n_y = 50$ 的误差云图

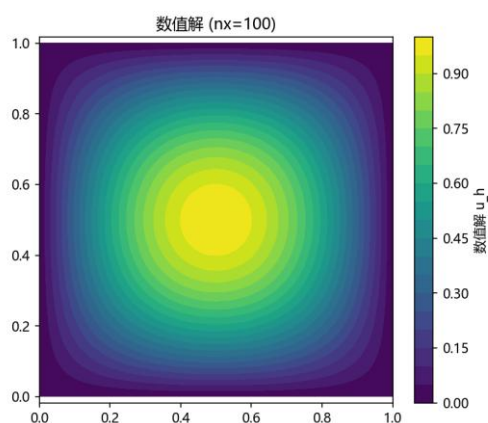


图 12. $n_x = n_y = 100$ 的数值解云图

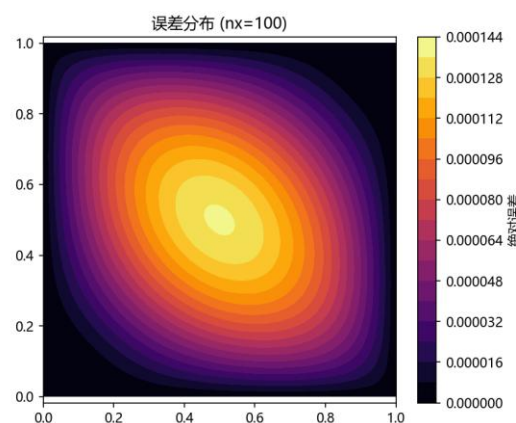


图 13. $n_x = n_y = 100$ 的误差云图

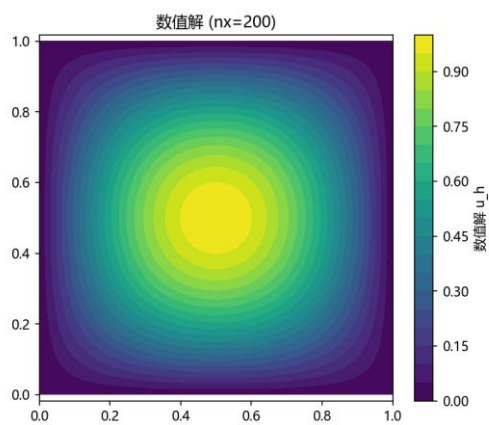


图 14. $n_x = n_y = 200$ 的数值解云图

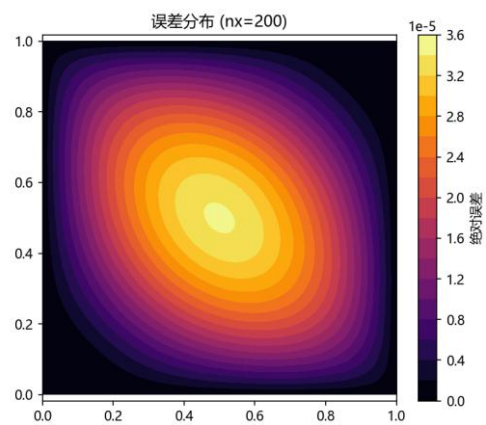


图 15. $n_x = n_y = 200$ 的误差云图

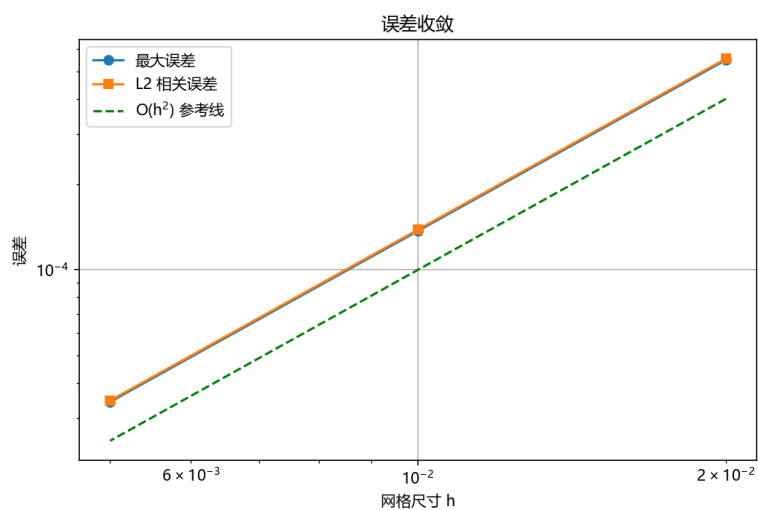


图 16. 误差随网格尺寸变化图

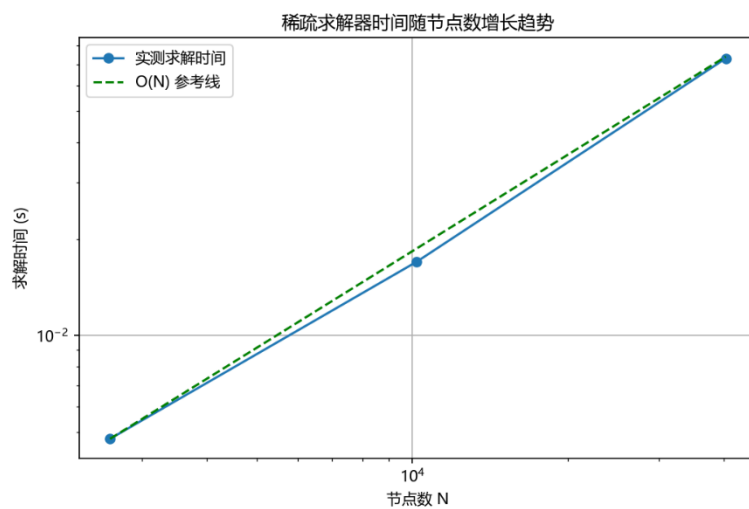


图 17. 稀疏求解器求解时间随节点数增长趋势图

4.8.3 误差收敛分析

- (1) 由图 10~15 数值解云图与理论解 $\sin(\pi x)\sin(\pi y)$ 形状一致，误差云图显示最大误差出现在中心区域，边界处误差接近零。
- (2) 由图 16 可知，最大误差和 L2 误差随网格加密而下降，当 h 减半时，误差约减小为原来的 1/4，验证了线性三角形单元的 $O(h^2)$ 收敛阶。
- (3) 由图 17 可知，稀疏求解时间随节点数增长近似线性，体现了稀疏求解器的效率。

第5章 误差与效率分析讨论

5.1 直接解法和迭代法

5.1.1 直接解法 (LDL^T)

对于中小规模问题 ($n \leq 10^4$)，计算稳定，结果精确。但内存需求为 $O(n^2)$ ，计算量 $O(n^3)$ ，无法处理百万自由度问题。

5.1.2 稀疏求解器 (Intel MKL PARDISO)

内存需求与 $O(nz)$ 成正比 (通常 $nz \propto n$)，计算量约 $O(n^{1.5})$ ，适合大规模稀疏问题。

5.1.3 迭代解法 (如共轭梯度法)

内存需求极小，计算量 $O(n \cdot nz)$ ，适合超大规模问题，但收敛速度和精度依赖于预处理子和条件数。

5.2 病态问题中“残差小但不准确”的解释

由误差分析公式：

$$\frac{\|\delta a\|}{\|a\|} \leq \text{cond}(K) \cdot \frac{\|r\|}{\|R\|} \quad (5-1)$$

相对误差上限=条件数×相对残差。条件数很大时，即使相对残差很小，相对误差也可能很大。因此，对于病态矩阵，必须同时关注条件数和残差，不能仅凭残差判断解的精度。

5.3 活动列/轮廓储存与缓存友好实现

LDL^T 分解的常规三重循环存在大量缓存未命中。通过活动列/轮廓存储，只存储矩阵的下三角部分，并利用列消去的递推关系，可以显著提高数据局部性，减少内存访问时间。

对于窄带宽矩阵（如三对角矩阵），活动列存储的浮点运算次数约为 $\frac{1}{2} \sum_{j=1}^n (j - m_j)^2$ ，其中 m_j 为第 j 列的第一个非零元行号，接近 $O(nb^2)$ （ b 为半带宽），极大提升了效率。

5.4 大规模有限元矩阵必须使用稀疏存储的原因

5.4.1 内存限制

稠密存储需要 n^2 个浮点数，对于 $n = 10^5$ ，需要 80GB 内存，远超一般计算机容量；而稀疏存储仅需 $O(nz)$ ，通常 $nz \approx 10n$ ，仅需几十 MB。

5.4.1 计算效率

稠密 LDL^T 需要 $O(n^3)$ 次浮点运算，对于 $n = 10^5$ 无法在可接受时间内完成；稀疏求解器利用矩阵稀疏性，计算量大幅降低。

第 6 章 总结

本次作业完成了以下工作：

1. 独立实现了对称正定矩阵的 LDL^T 分解和求解器（`ldlt_factor`, `ldlt_solve`），不依赖任何现成求解库，并包含非正主元检测。
2. 通过 JSON 文件实现了与 2.3 作业的无缝衔接，完整复用了已有的有限元建模和组装代码。2.3 作业负责“组装方程”，2.4 作业负责“高效求解方程”。
3. 验证了多个算例：
 - （1）2.3 桁架算例（正确求解位移、约束反力、单元应力/轴力，位移与理论值完全一致）。
 - （2）三对角矩阵性能测试（记录了不同规模下的求解时间，验证了 $O(n^3)$ 复杂度）。
 - （3）非正定矩阵检测（正确报错）。
 - （4）病态矩阵误差分析（展示了残差小但解不准的现象，解释了条件数的重要性）。

- (5) 多载荷工况效率对比（验证了“先分解再求解多个右端项”的效率优势）。
 - (6) 大规模稀疏求解器调用（对比了稀疏存储与稠密存储的内存差异，验证了稀疏求解器的效率）。
 - (7) 二维 Poisson 方程有限元求解（完成了网格生成、单元矩阵组装、边界条件处理、稀疏求解、误差分析和云图绘制，验证了 $O(h^2)$ 收敛阶）。
4. 讨论了直接解法与迭代解法的适用场景、病态问题的误差放大机制、稀疏存储的必要性和缓存友好的活动列存储原理。